



Informe

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

14 de Junio de 2026

1. Equipo	2
2. Introducción	2
3. Modelo Computacional	2
3.2.1. Definición de Autómatas	3
3.2.2. Modificación de Autómatas	4
3.2.3. Ejecución y Testeo	4
3.2.4. Conversiones	4
3.2.5. Visualización e Inspección	4
3.2.6. Tipos de Datos y Control	4
4. Implementación	5
4.2.1. $AFN \rightarrow AFD$ (convertNFAtoDFA)	6
4.2.2. $AFN-\lambda \rightarrow AFN$ (convertLNFAtoNFA)	7
4.2.3. $AFN-\lambda \rightarrow AFD$ (convertLNFAtoDFA)	8
4.2.4. $AFD \rightarrow AFN$ (convertDFAtoNFA)	8
4.2.5. Equivalencia	9
5. Futuras Extensiones	9
6. Conclusiones	9
7. Bibliografía	10

1. Equipo

Nombres	Apellidos	Legajos	E-mails
Carlos Amador	Vallejo Tapia	65454	cvallejotapia@itba.edu.ar
Bautista Santiago	Simone	65516	bsimone@itba.edu.ar
Jaime	Nazar Anchorena	65064	jnazaranchorena@itba.edu.ar
Lucas	Desmery	65663	ldesmery@itba.edu.ar

2. Introducción

El presente trabajo consistió en el desarrollo de un DSL para definir y operar sobre autómatas, con el objetivo de profundizar los temas vistos en la primera parte de la materia. Se desarrolló un compilador utilizando el lenguaje de programación C y herramientas como Flex y Bison. El código fue versionado en un repositorio de github y el desarrollo se dividió en tres etapas.

En la primera etapa del trabajo se confeccionó un documento de especificación, detallando el lenguaje a implementar y sus posibles casos de uso. Luego, en la segunda etapa se llevó a cabo el desarrollo del frontend mediante el uso de Flex y Bison, analizador léxico y sintáctico respectivamente. Finalmente se implementó el backend, encargado de la generación de código, la cual se trata de un archivo en formato Markdown.

3. Modelo Computacional

Para la implementación del compilador, se basó en dos aspectos fundamentales: Dominio y Lenguaje. Estos fueron especificados en la primera etapa del proyecto, antes de comenzar con el desarrollo.

3.1. Dominio

Durante la primera parte de la materia, se requiere realizar varias operaciones sobre autómatas y desarrollar una clara comprensión del tema. La manera más común de verificar que un ejercicio haya sido completado correctamente es compararlo con la respuesta provista por la cátedra o consultar a un docente. De manera que surge la necesidad de poder verificar la correcta resolución de ejercicios de manera más sencilla, por lo que se buscó desarrollar un lenguaje que permita realizar operaciones con autómatas finitos, ya sea deterministas, no deterministas o no deterministas lambda. Dicho lenguaje permitirá evaluar si una palabra es

aceptada o no, determinar si dos autómatas son equivalentes, y realizar la conversión entre los distintos tipos.

La correcta implementación del mismo permitirá reducir el trabajo mecánico que requiere operar con autómatas, principalmente los algoritmos de conversión. Además, permitirá mostrar, mediante la práctica, el comportamiento de los autómatas finitos y cómo se comparan entre sí. Se trata de un dominio específico a operaciones con autómatas finitos.

3.2. Lenguaje

El lenguaje desarrollado está inspirado en el lenguaje Rust, presenta construcciones del siguiente estilo:

```
// Crear autómata DFA
automaton ParidadCeros : DFA {
    alphabet      = {0, 1}
    states        = {q0, q1}
    start         = q0
    accept        = {q0}
    transitions {
        q0 -> 0 : q1
        q0 -> 1 : q0
        q1 -> 0 : q0
        q1 -> 1 : q1
    }
};

// Editar transiciones existentes
ParidadCeros {
    transitions {
        q0 -> 1 : q1    // Sobreescribe la transición q0->1
    }
};

test ParidadCeros with "0011";
test ParidadCeros with "010";
test ParidadCeros with "";
```

Donde se puede observar como se define un autómata, edita y se verifica si acepta una cadena o no. En las siguientes subsecciones se describe de una manera más específica.

3.2.1. Definición de Autómatas

- Se podrá crear un autómata de tipo AFD, AFN o AFNL especificando alfabeto, estados, estado inicial y estados de aceptación.

- Se podrán declarar transiciones con sintaxis `q0 -> símbolo : destino o conjuntos`
`q0 -> símbolo : {q1, q2}`
- Las transiciones lambda se declaran con la palabra reservada `lambda` como símbolo.
- Se podrán crear autómatas vacíos (sin ninguna transición) para completar luego.

3.2.2. Modificación de Autómatas

- Se podrán agregar o eliminar transiciones de un autómata ya definido usando la sintaxis de bloque `NombreAutomata { transitions { ... } }`
- Se podrán agregar nuevos estados y redefinir el conjunto de aceptación.

3.2.3. Ejecución y Testeo

- Se podrá testear una cadena sobre un autómata con `test <A> with "cadena"`. Devuelve `accept` o `reject`.
- Se soporta la cadena vacía como entrada (`""`).
- La ejecución sobre AFNL calcula automáticamente la clausura- λ en cada paso.

3.2.4. Conversiones

- Se podrá convertir un autómata a otro tipo con `convert <A> to <tipo> as <nombre>`.
- Conversiones soportadas:
 - $AFN \rightarrow AFD$ (subconjuntos)
 - $AFN-\lambda \rightarrow AFN$ (eliminación de λ)
 - $AFN-\lambda \rightarrow AFD$ (directa)
 - $AFD \rightarrow AFN$ (trivial, agrega no-determinismo vacío)

3.2.5. Visualización e Inspección

- Se podrá imprimir la definición de un autómata con `print <A>`.
- Se podrá mostrar la tabla de transición con `show table of <A>`.
- Se podrá mostrar la clausura- λ de un estado con `show closure(<q>) in <A>`.
- Se podrá mostrar las transiciones de un autómata con `show transitions of <A>`.

3.2.6. Tipos de Datos y Control

- Las variables pueden ser de tipo `Automaton` o `String`.
- Se proveerá una estructura de control para iterar: `for`

4. Implementación

4.1. Frontend

Como se mencionó anteriormente, el frontend fue desarrollado con Flex y Bison. El analizador léxico se utilizó para generar los tokens de la primera parte del frontend. Luego, se utilizó el analizador sintáctico para realizar chequeos sobre la gramática, pero no todos los chequeos sobre el lenguaje. Por ejemplo, se puede verificar que la definición de autómata esté bien estructurada, pero no si tiene sentido lógico:

```
// Crear autómata DFA con transiciones lambda
automaton AutomataInvalido : DFA {
    alphabet      = {0, 1}
    states        = {q0, q1}
    start         = q0
    accept        = {q0}
    transitions {
        q0 -> lambda : q1
        q0 -> 1 : q0
        q1 -> 0 : q0
        q1 -> 1 : q1
    }
};
```

Por otro lado, se buscó crear una gramática no ambigua que sea lo más modular posible, es decir, que se puedan reutilizar producciones para otras estructuras, reduciendo el código repetido y mejorando la lectura del código.

4.2. Backend

Similarmente al frontend, cuenta con dos partes, el generador de código y la verificación de la correcta semántica, logrando así validar cosas que no eran posibles de validar en el frontend (ejemplo mencionado anteriormente).

En primer lugar, se procesan los autómatas definidos en el frontend y se insertan en la tabla de símbolos. En términos del alcance de cada variable declarada, se cambia el alcance al entrar en un bloque de código como en el caso de un for. De manera que no solo hay un único scope de símbolos, sino que puede existir más de uno.

Por otro lado, se definen los métodos encargados de realizar las operaciones sobre autómatas, como el testeo, pasaje de uno a otro o imprimir un autómata. Además, se realizó la validación de tipos sobre las definiciones de los autómatas como tener un símbolo repetido en el alfabeto, falta de un estado que aparece en las transiciones o simplemente usar el mismo nombre para dos variables distintas.

Para el backend se introdujeron nuevos tipos de datos propios de esta capa, cuya existencia responde a una decisión de diseño deliberada: desacoplar la representación sintáctica del árbol de sintaxis abstracta (AST) de la representación de ejecución. El tipo `Automaton`, definido en el frontend, refleja fielmente lo que escribió el usuario en el código fuente: conserva la estructura tal como fue parseada, con sus listas de estados, transiciones y destinos en la forma literal en que aparecen en el programa. Este tipo es, en esencia, inmutable y pertenece exclusivamente al análisis léxico-sintáctico.

En contraste, el backend opera sobre `RuntimeAutomaton`, un tipo pensado para la ejecución. A diferencia de `Automaton`, `RuntimeAutomaton` utiliza una representación normalizada y uniforme que facilita la implementación de los algoritmos: por ejemplo, los destinos de cada transición siempre se representan como una lista (`RuntimeStringList`), eliminando el union de destino único versus múltiple que existe en el AST. Esta uniformidad simplifica considerablemente el código de simulación y conversión. Además, las instancias de `RuntimeAutomaton` pueden crearse, clonarse y destruirse libremente durante la ejecución, como ocurre con el autómata intermedio generado en la conversión de AFN- λ a AFD.

El prefijo `Runtime` en todos estos tipos (`RuntimeAutomaton`, `RuntimeTransition`, `RuntimeStringList`, `RuntimeSymbol`, `RuntimeScope`, `RuntimeSymbolTable`) marca visualmente la pertenencia a la capa de ejecución y distingue de un vistazo las estructuras del parser de las que viven durante la interpretación del programa.

Y por último, la generación de código, la cual se encarga de generar un archivo `.md` donde se describen las operaciones realizadas y los resultados obtenidos. Utiliza lo obtenido en la primera etapa para generar el código garantizando que es correcto.

4.2.1. AFN $\rightarrow$ AFD (`convertNFAtoDFA`)

Para este pasaje se utiliza el algoritmo de construcción de subconjuntos. La idea es ir agrupando en conjuntos los posibles estados a los que puede moverse el autómata no determinista al consumir un símbolo, y tratar cada conjunto como un único estado en el AFD resultante.

Se define una estructura `SubsetEntry` que representa un estado del AFD en construcción. Cada entrada contiene un conjunto de estados del AFN original (`states`), el nombre generado para ese estado (`name`), y un flag `processed` que indica si ya se calcularon sus transiciones:

```
typedef struct SubsetEntry {
    RuntimeStringList * states;
```

```
char * name; //ej: "{q0,q1}"
bool processed;
struct SubsetEntry * next;
} SubsetEntry;
```

Para poder identificar cada subconjunto sin repetirlos, se usa la función `_buildSubsetName` que clona la lista de estados, la ordena alfabéticamente con `_sortStringList` (un bubble sort simple), y concatena los nombres entre llaves. Por ejemplo, si el conjunto contiene `{q2, q0, q1}`, el nombre generado será `"{q0,q1,q2}"`.

El algoritmo implementado funciona de la siguiente manera:

1. Se crea el AFD vacío con el mismo alfabeto que el AFN.
2. El estado inicial del AFD es igual al estado inicial del AFN y se crea un `SubsetEntry` con ese único estado.
3. Se entra en un bucle que busca entradas no procesadas y para cada una:
 - Se itera sobre cada símbolo del alfabeto.
 - Se llama a `_moveNFA(nfa, current->states, sym->value)` que recorre las transiciones del AFN y recolecta todos los destinos posibles desde los estados del conjunto actual con ese símbolo, sin duplicados gracias a `appendUniqueRuntimeString`.
 - Si el resultado no es vacío, se busca con `_findSubset` si ya existe un `SubsetEntry` con ese mismo conjunto de estados, comparando con `_sameStringSet` que verifica igualdad de conjuntos.
 - Si no existe, se crea un nuevo `SubsetEntry` y se agrega a la lista.
 - Se crea una `RuntimeTransition` desde el subconjunto actual al subconjunto destino.
4. Al terminar el bucle, se recorren todos los `SubsetEntry` y se marca como estado de aceptación a todo subconjunto que contenga al menos un estado de aceptación del AFN original (`runtimeStringListContains` sobre `nfa->acceptStates`).
5. Se libera la lista de `SubsetEntry` con `_destroySubsetList`.

4.2.2. $AFN-\lambda \rightarrow AFN$ (`convertLNFAtoNFA`)

En el pasaje de autómatas no deterministas finitos lambda a autómatas no deterministas se utiliza la clausura lambda para identificar qué estados son alcanzables sin consumir ningún símbolo, y así construir las transiciones equivalentes sin lambdas.

La función `lambdaClosure` recibe un autómata y un conjunto de estados iniciales, y devuelve todos los estados alcanzables desde ese estado inicial siguiendo únicamente transiciones lambda. Se implementa como un algoritmo que empieza con los estados iniciales en la clausura, e

itera recorriendo las transiciones lambda desde cada estado de la clausura. Si se encuentra un destino que no está en la clausura, se agrega y se cambia `changed` a `true` para repetir el ciclo. El bucle termina cuando no se agregan más estados.

El algoritmo implementado funciona de la siguiente manera:

1. Se crea el AFN vacío, copiando el alfabeto, los estados y el estado inicial del AFN- λ .
2. Se recorre cada estado del AFN- λ , se calcula su clausura lambda, llamando a `lambdaClosure`, y si la clausura contiene algún estado de aceptación original, ese estado se marca como de aceptación en el AFN resultante.
3. Nuevas transiciones: para cada estado q del AFN- λ :
 - Se calcula $\text{clausura-}\lambda(\{q\})$.
 - Para cada símbolo a del alfabeto, se llama a `_moveNFA` con la clausura y el símbolo para obtener los estados alcanzables.
 - Si `moved` no es vacío, se calcula la clausura- λ para cerrar con lambdas nuevamente.
 - Se crea una `RuntimeTransition` desde q con símbolo a hacia todos los estados de la clausura del movimiento, (agregados con `appendUniqueRuntimeString` para evitar duplicados).
4. Se libera la memoria de las clausuras intermedias en cada iteración.

4.2.3. AFN- $\lambda \rightarrow$ AFD (`convertLNFAtoDFA`)

Este pasaje es directo teniendo los dos anteriores ya implementados. Simplemente se aplica primero la conversión de AFN- λ a AFN (`convertLNFAtoNFA`) y luego la de AFN a AFD (`convertNFAtoDFA`) sobre el resultado del primer pasaje:

El autómata intermedio se crea con el nombre temporal "`__intermediate__`" y se destruye con `destroyRuntimeAutomaton` una vez que el AFD final está construido, para no dejar memoria sin liberar.

4.2.4. AFD $\rightarrow$ AFN (`convertDFAtoNFA`)

Este caso es trivial, ya que todo autómata finito determinista es a su vez un autómata no determinista. Por lo tanto, solo es necesario realizar una copia profunda del autómata y cambiarle el tipo:

La función `cloneRuntimeAutomaton` se encarga de copiar todos los componentes: alfabeto, estados, estado inicial, estados de aceptación y transiciones (incluyendo cada string con

copyRuntimeString y cada lista de destinos). El resultado es un autómata idéntico pero con type = NFA.

4.2.5. Equivalencia

En la implementación de la funcionalidad de verificar que dos autómatas son equivalentes, se tomó la decisión de utilizar el algoritmo de exploración de pares de estado, creado por John Hopcroft y Richard Karp en 1971 en el paper “A linear algorithm for testing equivalence of finite automata”. Se optó por este algoritmo por sobre la alternativa de minimizar ambos autómatas con el algoritmo de minimización de John Hopcroft ya que buscando información sobre cómo desarrollar esta funcionalidad, se encontró que era la alternativa más eficaz de ambas. La base del algoritmo está en recorrer los nodos de ambos autómatas al mismo tiempo comenzando por el par (q_0, p_0) y avanzando símbolo a símbolo por ambos autómatas al mismo tiempo, como si los ejecutará en sincronía. A cada movimiento posible se revisa si ambos nodos a los que se esta son finales para determinar si la cadena hasta ahí se acepta o no en ambos. La primera vez que uno esté en estado final y otro no, ahí se considera que no son equivalentes y se devuelve falso.

4.3. Dificultades encontradas

Una dificultad encontrada fue la necesidad de implementar un scope para los bloques de código correspondientes a los for. Esto se debe a que es necesario diferenciar cada scope en la tabla de símbolos, complejizando la implementación.

5. Futuras Extensiones

En base a los avances vistos en la segunda parte de la materia, se consideró extender el lenguaje desarrollado con el agregado de autómatas de pila y máquinas de turing. Esto se debe a que además de ser avances del material teórico también aprovecha de la modularidad del lenguaje ya creado, facilitando su extensión. Por otro lado, también se consideró agregar operaciones como la unión y la concatenación,

6. Conclusiones

En síntesis, durante el desarrollo de este trabajo práctico especial se diseñó e implementó un compilador para un DSL con la ayuda de herramientas como Flex y Bison. Se aplicó la teoría vista a lo largo de la cursada, para el frontend del compilador se utilizó el análisis léxico y sintáctico, mientras que para el backend se aplicó el análisis semántico y la generación de código.

Por otro lado, por la naturalidad del trabajo se puso en práctica la resolución de problemas y el trabajo en equipo mediante el uso de github para el versionado de código y se aprovechó de tests

de integración continua proveídos por el repositorio base para garantizar buenas prácticas de desarrollo.

7. Bibliografía

- ☰ (2025-09-03, v1.0.0) Análisis Léxico
- ☰ (2024-05-08, v0.1.0) Análisis Sintáctico
- ☰ (2024-09-25, v0.2.0) Análisis Semántico
- ☰ (2024-05-16, v0.1.0) Generación de Código
- ☰ (2024-05-16, v0.1.0) Runtime